

Assignment – Inference Engineering

1. Why is inference memory bandwidth-bound important?

Inference memory bandwidth is important because LLMs need to continuously move large amounts of model weights and other data from GPU memory to the compute units.

- If memory bandwidth is low, the GPU may spend time waiting for data even when its computing power is high.
- Higher memory bandwidth allows data to be supplied to the GPU faster, improving token generation speed and throughput.
- Therefore, memory bandwidth is an important factor when selecting GPUs and optimizing LLM inference.

2. How can we calculate the overall memory required to store a model? How are LLM models represented while storing?

An LLM is mainly stored as a collection of numerical values called **parameters or weights**. These weights are represented using different numerical precisions such as FP32, FP16, BF16, INT8, and INT4.

The basic formula for calculating model weight memory is:

$$\text{Memoryrequired} = \text{Numberofparameters} \times \text{Bytesperparameter}$$

For example, a 7-billion-parameter model stored in FP16 requires approximately:

$$7B \times 2\text{bytes} = 14GB$$

The approximate memory required for common precisions is:

- FP32 = 4 bytes per parameter
- FP16/BF16 = 2 bytes per parameter
- INT8 = 1 byte per parameter
- INT4 = 0.5 byte per parameter

However, during inference, total GPU memory is more than just model weights. It also includes KV cache, activations, temporary buffers, and runtime overhead.

$$\text{Totalinferencememory} \approx \text{Modelweights} + \text{KVcache} + \text{Runtimeoverhead}$$

Therefore, the calculated weight memory should be considered a theoretical minimum.

3. Explain each layer of inferencing and its importance to an institution/company.

LLM inference can be divided into three major layers:

A. Inference Runtime Layer

Purpose: This layer actually runs the LLM and generates the output.

Components:

- **PyTorch** – Deep learning framework used to run AI models.
- **CUDA** – NVIDIA software platform that allows applications to use GPUs.
- **vLLM** – Efficient LLM inference and serving framework.
- **SGLang** – Framework for efficient LLM serving and complex workflows.
- **TensorRT-LLM** – NVIDIA framework for optimizing LLM inference.

Importance: It improves inference speed, GPU utilization, throughput, and resource efficiency.

B. Infrastructure Layer

Purpose: This layer manages the servers, GPUs, traffic, and resources required to run inference.

Components:

- **Kubernetes** – Manages containers and inference servers.
- **Load Balancer** – Distributes user requests among available servers.
- **Autoscaler** – Automatically increases or decreases resources according to demand.
- **Capacity Pools** – Groups of available GPUs or computing resources.

Importance: It provides scalability, reliability, and efficient management of computing resources.

C. Tooling Layer

Purpose: This layer provides tools to deploy, configure, manage, and monitor inference systems.

Components:

- **Deployment APIs** – Used to deploy and manage models programmatically.
- **Config Schemas** – Define valid deployment configurations.

- **SDKs** – Make APIs easier for developers to use.
- **Dashboards** – Display metrics such as latency, throughput, GPU utilization, and errors.

Importance: It makes deployment, monitoring, and management easier and reduces manual work.

4. GPU Hardware Exploration

For comparison, two modern NVIDIA GPUs are selected: **NVIDIA B200** and **NVIDIA B300**.

GPU	VRAM	Bandwidth	Approx. Rental Cost
NVIDIA B200	180 GB HBM3e	8 TB/s	\$6.79/GPU-hour
NVIDIA B300	288 GB HBM3e	8 TB/s	\$7.89/GPU-hour

The rental prices are approximate and can vary depending on the cloud provider, region, and availability.

NVIDIA B200

The B200 provides approximately 180 GB of HBM3e memory and up to 8 TB/s memory bandwidth.

At 16-bit precision:

$$\frac{180GB}{2bytes/parameter} \approx 90Bparameters$$

At 4-bit precision:

$$\frac{180GB}{0.5bytes/parameter} \approx 360Bparameters$$

Therefore, theoretically, the B200 can store approximately a **90B parameter model at 16-bit** or a **360B parameter model at 4-bit**, considering only model weights.

NVIDIA B300

The B300 provides approximately 288 GB of HBM3e memory and up to 8 TB/s memory bandwidth.

At 16-bit precision:

$$\frac{288GB}{2bytes/parameter} \approx 144Bparameters$$

At 4-bit precision:

$$\frac{288GB}{0.5bytes/parameter} \approx 576Bparameters$$

Therefore, theoretically, the B300 can store approximately a **144B parameter model at 16-bit** or a **576B parameter model at 4-bit**, considering only model weights.

Note: These are theoretical maximum values based only on weight storage. Actual inference requires additional memory for KV cache, activations, and runtime overhead, so the practical model size will be smaller.

5. Serving Framework Exploration

A. Ollama

What it is: Ollama is a simple LLM runtime designed mainly for running models locally on laptops, PCs, and local servers.

Good for:

- Local development and testing.
- Quickly experimenting with different LLMs.
- Private applications where the model can run locally.

example: A student can run a local AI chatbot on their laptop without sending the conversation to a cloud server.

When to choose it: Ollama is preferred over vLLM, SGLang, or TensorRT-LLM when **simplicity and local development are more important than maximum production throughput**.

B. LMDeploy

What it is: LMDeploy is an LLM serving and deployment framework focused on efficient inference, quantization, and multi-GPU deployment.

Good for:

- High-throughput LLM inference.
- Quantized models such as INT4 and INT8.
- Multi-GPU deployment.
- Efficient management of KV cache and batching.

example: A company running a customer-support chatbot can use LMDeploy to serve a large model efficiently while using quantization to reduce memory requirements.

When to choose it: LMDeploy can be selected over vLLM, SGLang, or TensorRT-LLM when its **quantization support, model support, or inference optimizations better match the workload**.